2. 8-Puzzle using A* search and the Manhattan-distance heuristics.

```
8-PUZZLE-A-STAR (problem)
    node ← NODE(STATE(problem))
    frontier ← priority queue ordered by f(n)
    INSERT(frontier, node)
    explored ← empty set
    g(node) ← 0
    f(node) ← g(node) + MANHATTAN-DISTANCE(node.state)
    while frontier is not empty do
        node ← POP(frontier)
        if GOAL-TEST(problem, node.state) then
            return SOLUTION(node)
        if node.state not in explored then
            add node.state to explored
            for each action in ACTIONS(node.state) do
                child ← CHILD-NODE(problem, node, action)
                g(child) ← g(node) + 1
                f(child) ← g(child) + MANHATTAN-
                                DISTANCE(child.state)
                if child.state not in explored then
                    INSERT(frontier, child)
    return failure
```

```
CHILD-NODE (problem, node, action)
    state ← COPY (node.state)
    blank ← POSITION-OF-BLANK (state)
    if action = UP then
        move blank upward
    else if action = DOWN then
        move blank downward
    else if action = LEFT then
        move blank left
    else if action = RIGHT then
        move blank right
    Return NODE (
        STATE = state,
        PARENT = node,
        ACTION = action,
        PATH-COST = node.PATH-COST + 1 )

MANHATTAN-DISTANCE (state)
    distance ← 0
    for each tile t in state do
        if t is not blank then
            current ← POSITION (t, state)
            goal ← POSITION (t, GOAL-STATE)
            distance ← distance +
                [current.row - goal.row] +
                [current.column - goal.column]
    return distance
```

Test cases :-

1. One-move solution :

Input
1 2 3
4 5 6
7 _ 8

Output: move RIGHT
1 2 3
4 5 6
7 8 _

2. Two-move solution:

Input
1 2 3
4 _ 6
7 5 8

Output: move DOWN
1 2 3
4 5 6
7 _ 8

3. More complex solution:

Input
1 2 3
5 _ 6
4 7 8

move RIGHT
1 2 3
4 5 6
7 8 _

Output
1 2 3
4 5 6
7 8 _

Solution cost: 4